

Whiteboard diagrams from the course.

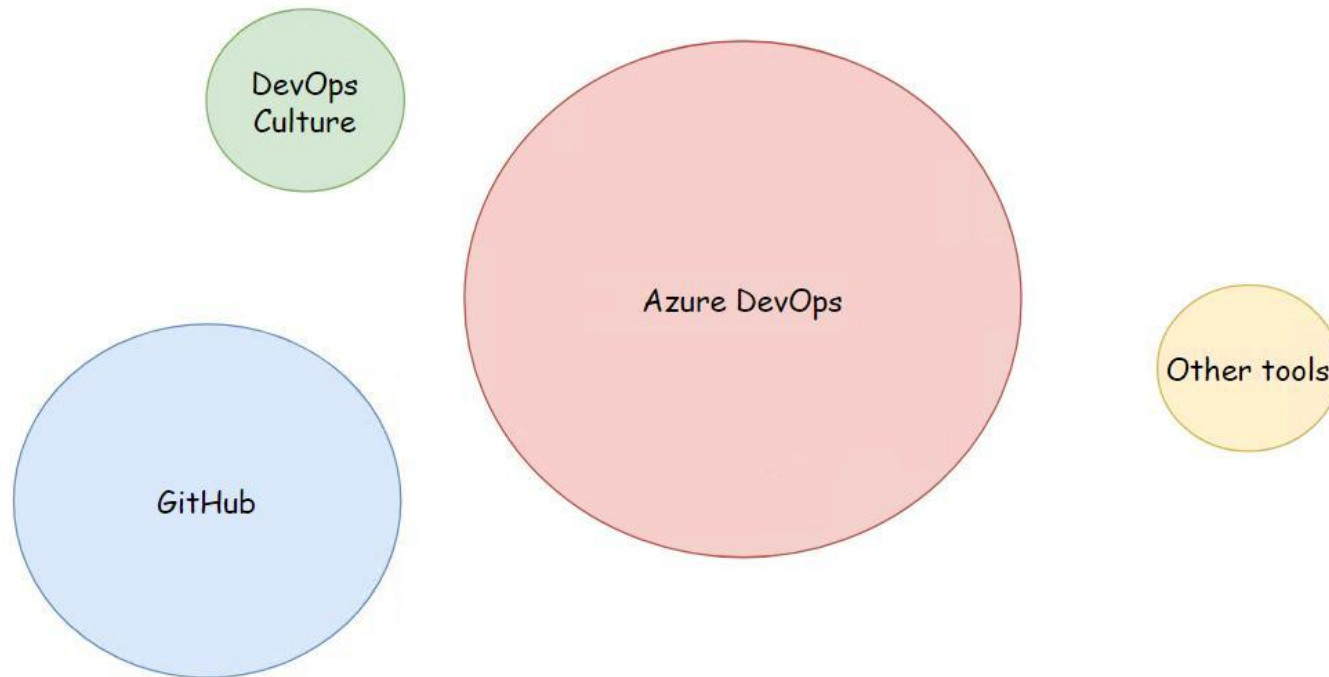
You can 'zoom in' to the diagrams, useful if the writing and diagram detail is small.

I'll add to this document as we go through the course.

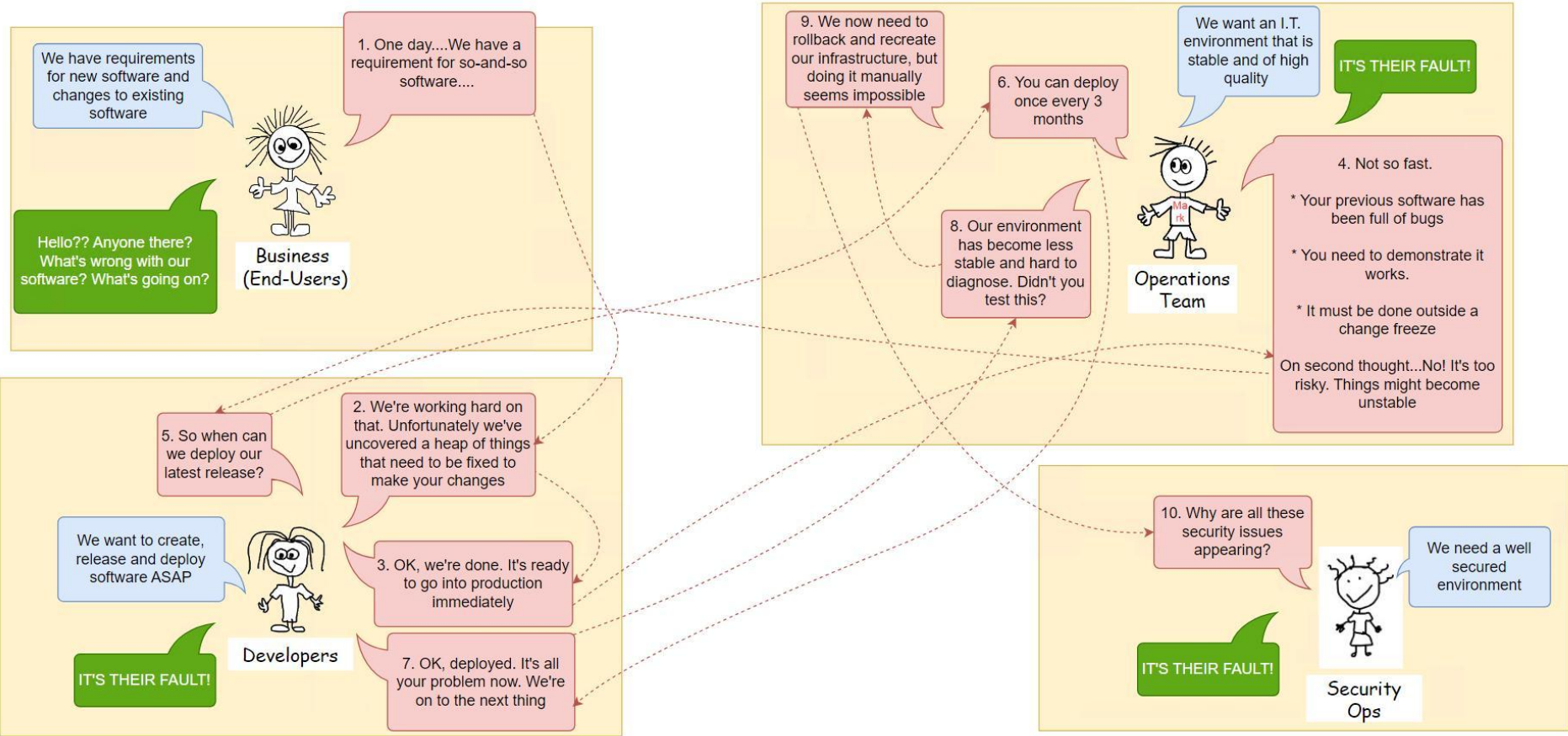
Mark.

My Description: I drew this diagram to summarise the key focus areas of the AZ-400 course. Most of the course is concerned with using the Azure DevOps set of products, there's a fair amount of GitHub material, a smaller section on DevOps culture to give the tools some context. Finally, a few other tools get a mention, most of which are used in conjunction with GitHub or Azure DevOps.

AZ-400 course focus areas

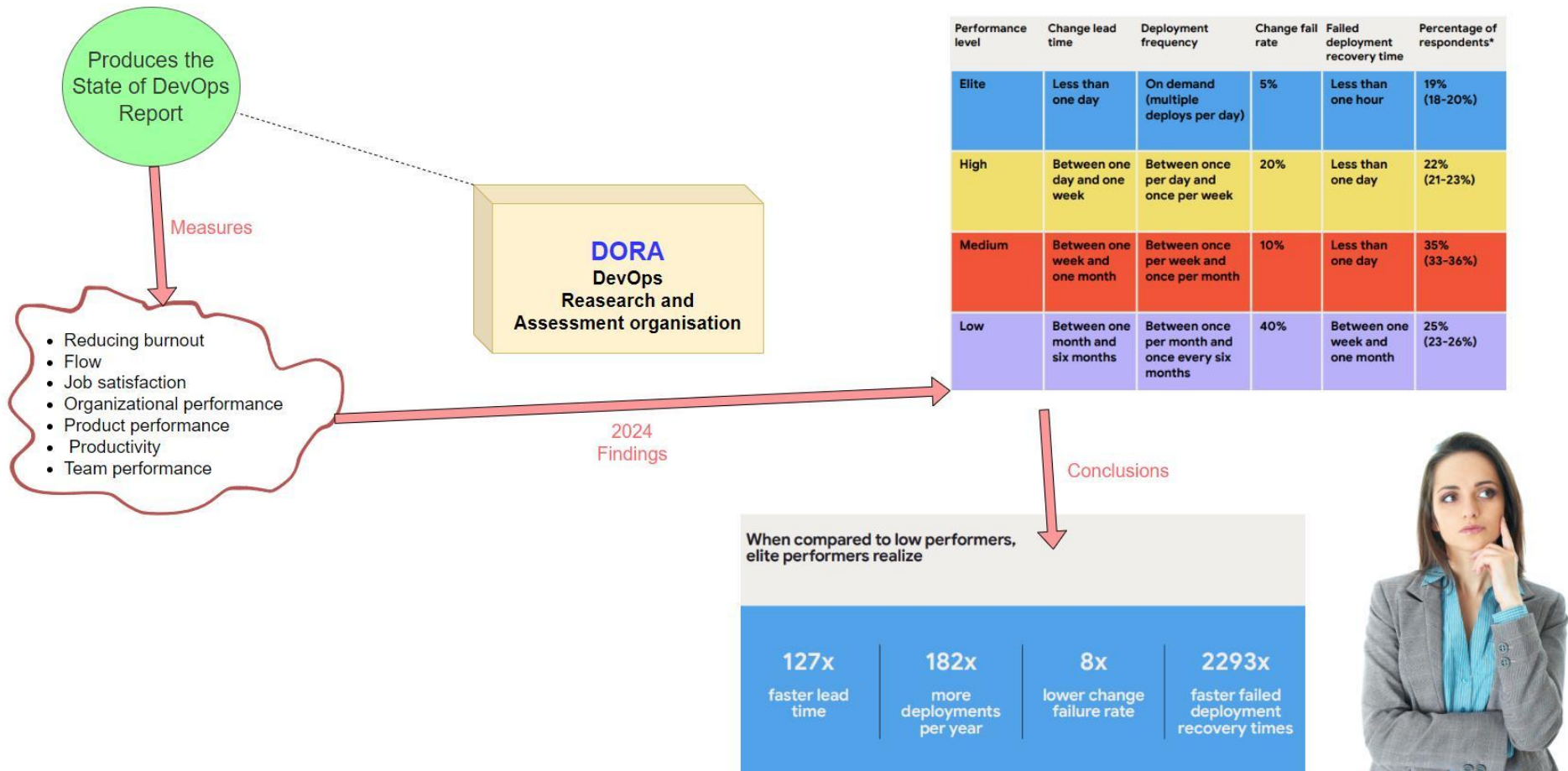


My Description: What usually happens with the development and delivery of software



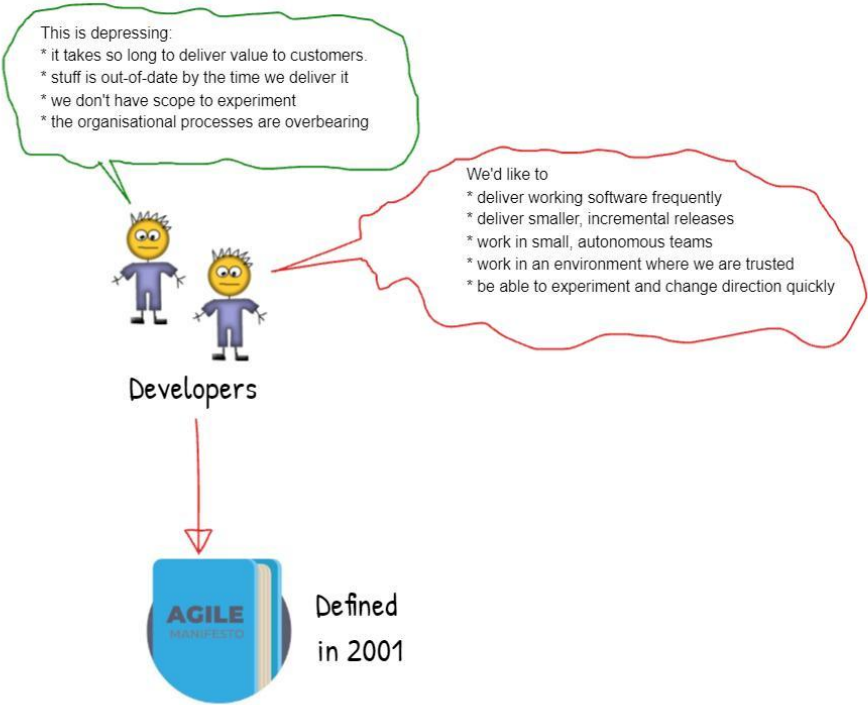
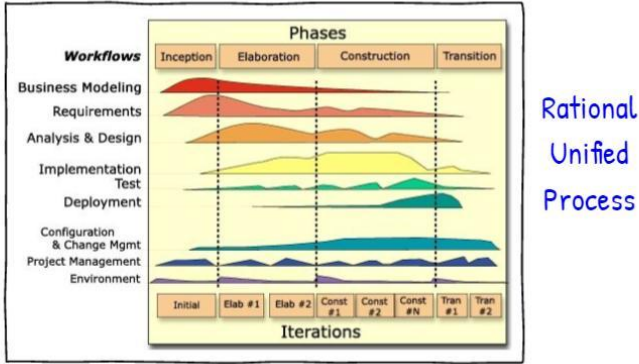
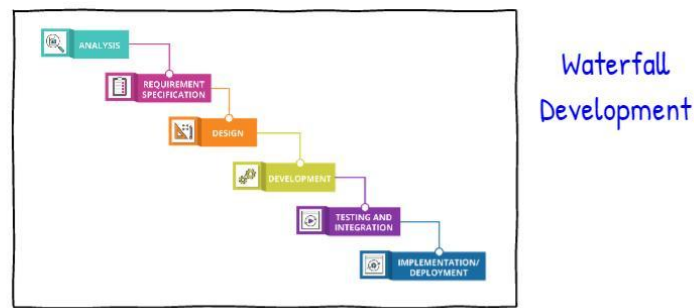
DiagramID: az-p-Traditional-IT-Dev-Cycle.

My Description: I put this in here so you might get an understanding about the adoption of DevOps culture and tooling across many enterprises. You might then work out where your organisation is on the journey.

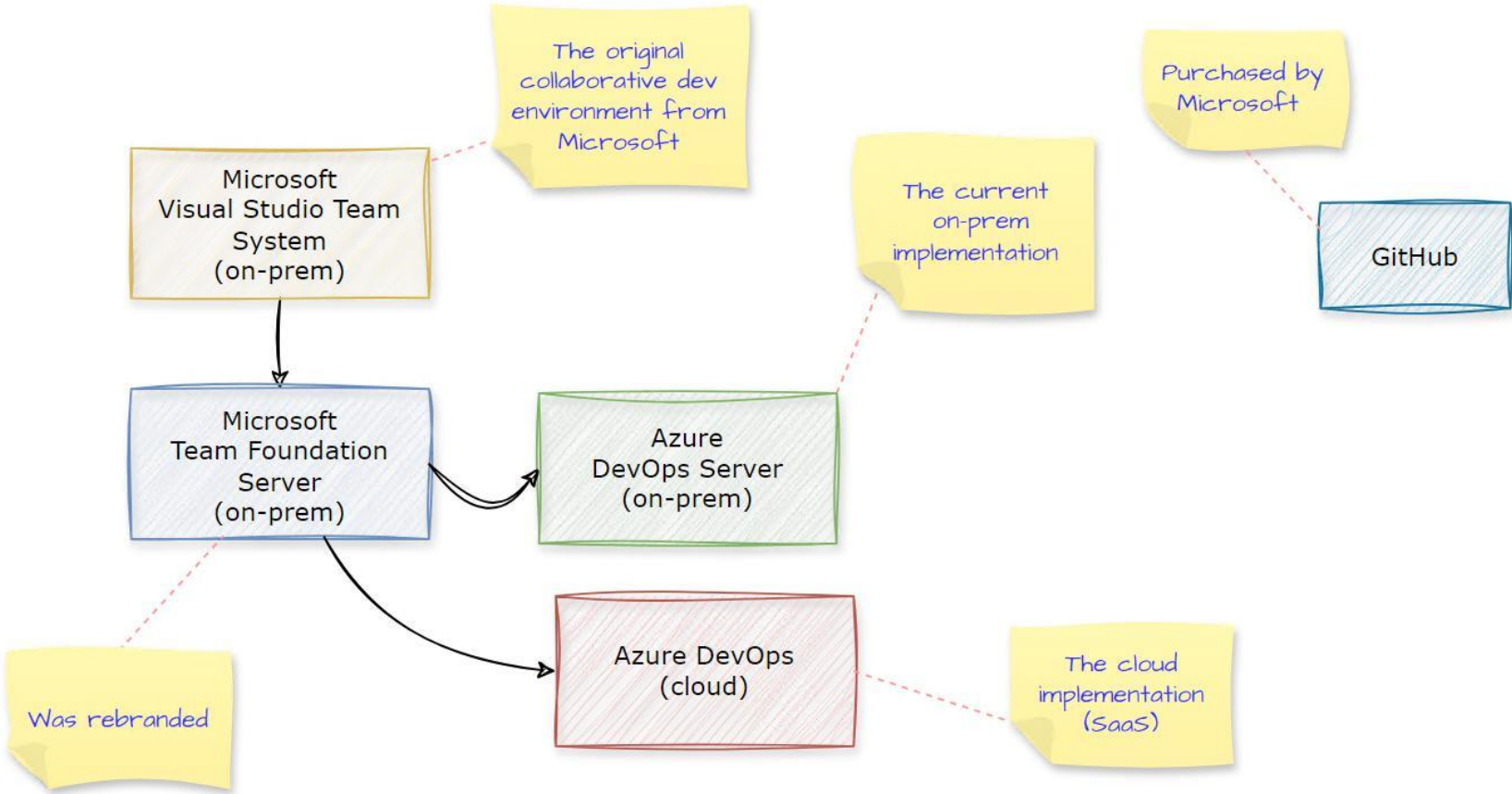


DiagramID: p-DORA-StateOfDevOps.

My Description: The history of Agile

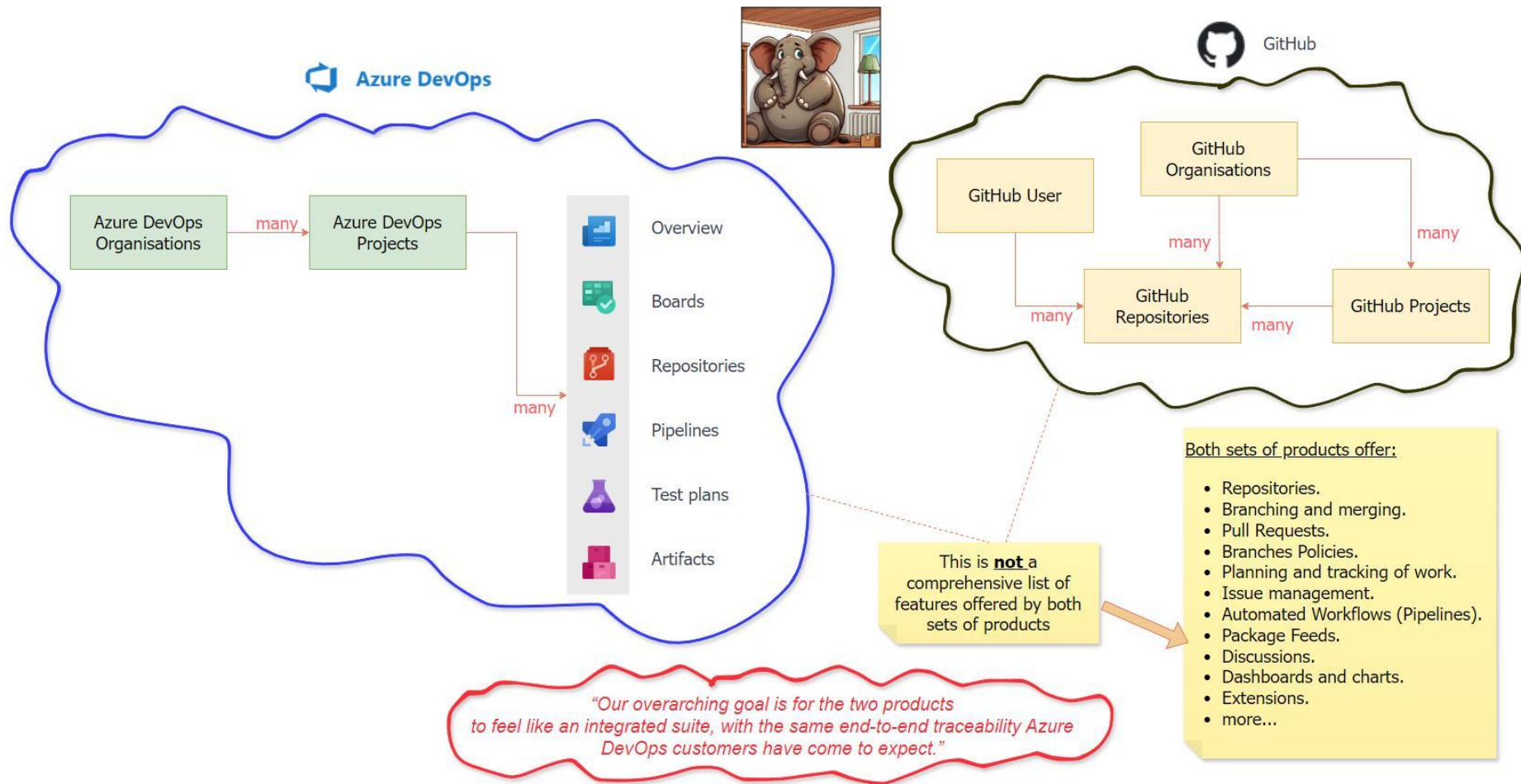


My Description: A diagram showing the history of Azure DevOps. It evolved from earlier products and has an on-prem installation and a SaaS implementation.

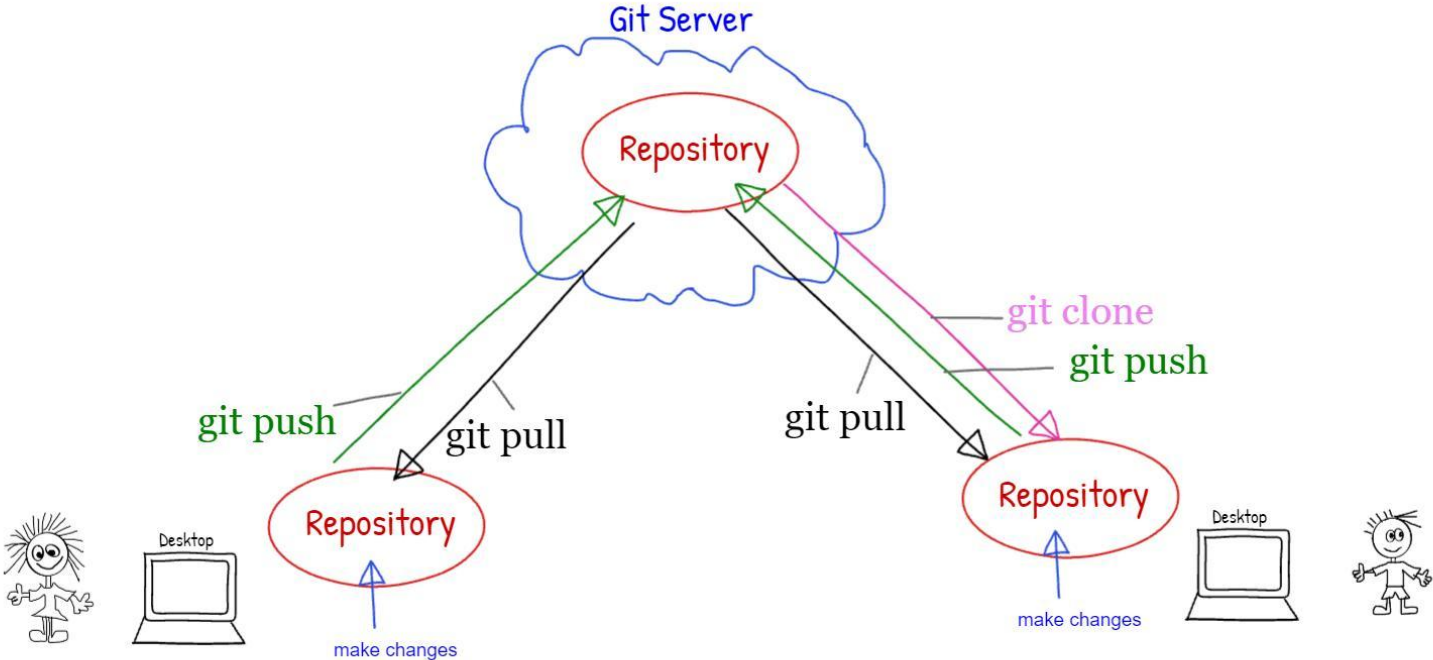


DiagramID: p-AzureDevOpsAndGithub.

My Description: You wouldn't be alone in being a little confused with Microsoft DevOps tooling directions. They essentially have two sets of tools, AzureDevOps is one set and GitHub the other. The AzureDevOps tools grew out of a set of products that Microsoft developed and were originally run on-premises. Github is a set of products they have purchased. At the time of writing it seems their aim is to keep both sets of products to keep both sets of customers. We will see cross-pollination of the two over time so they feel as if they are part 'of the same ecosystem'.



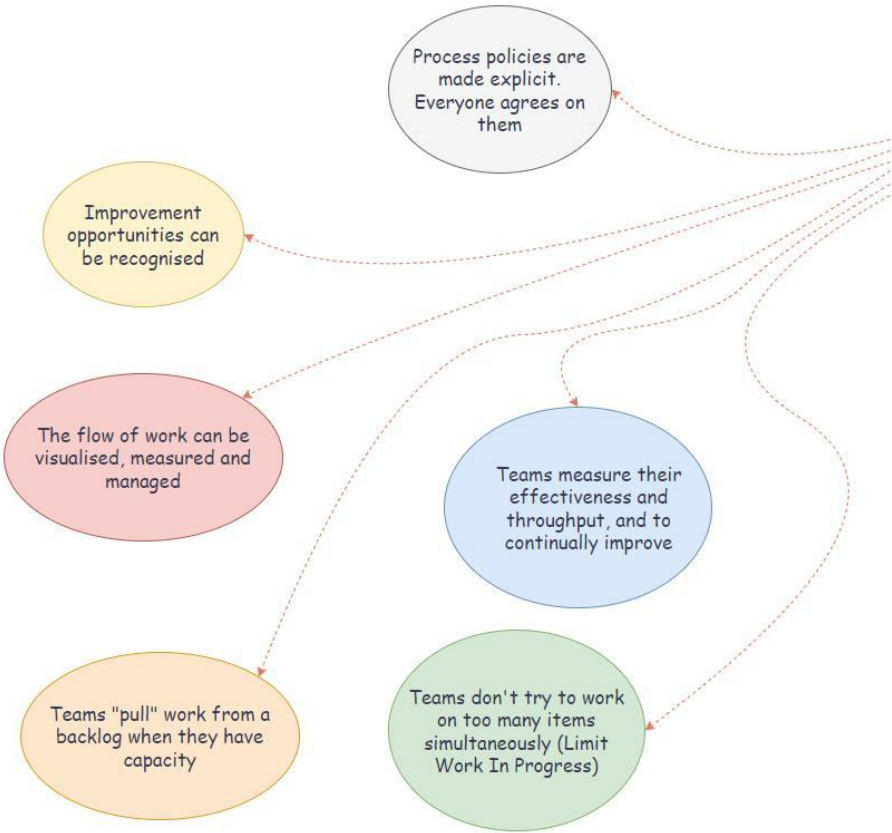
My Description: A diagram showing how multiple people can maintain their own copy of a repository and collaborate on changes made



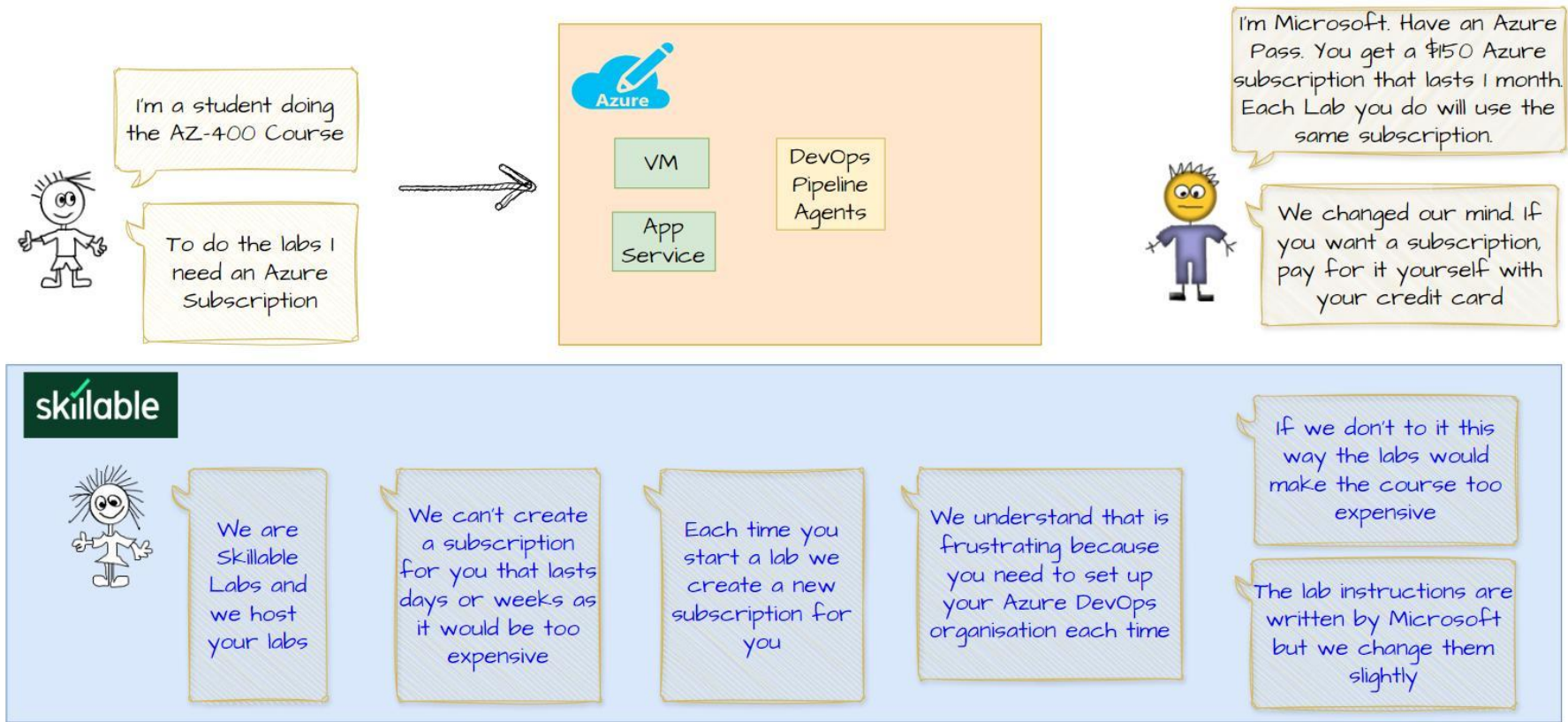
DiagramID: Git-Collaboration.

My Description: Kanban is not just Kanban boards. It's a methodology, originally applied to manufacturing but suitable to many situations and industries

Taiichi Ohno from the Toyota Corporation developed Kanban

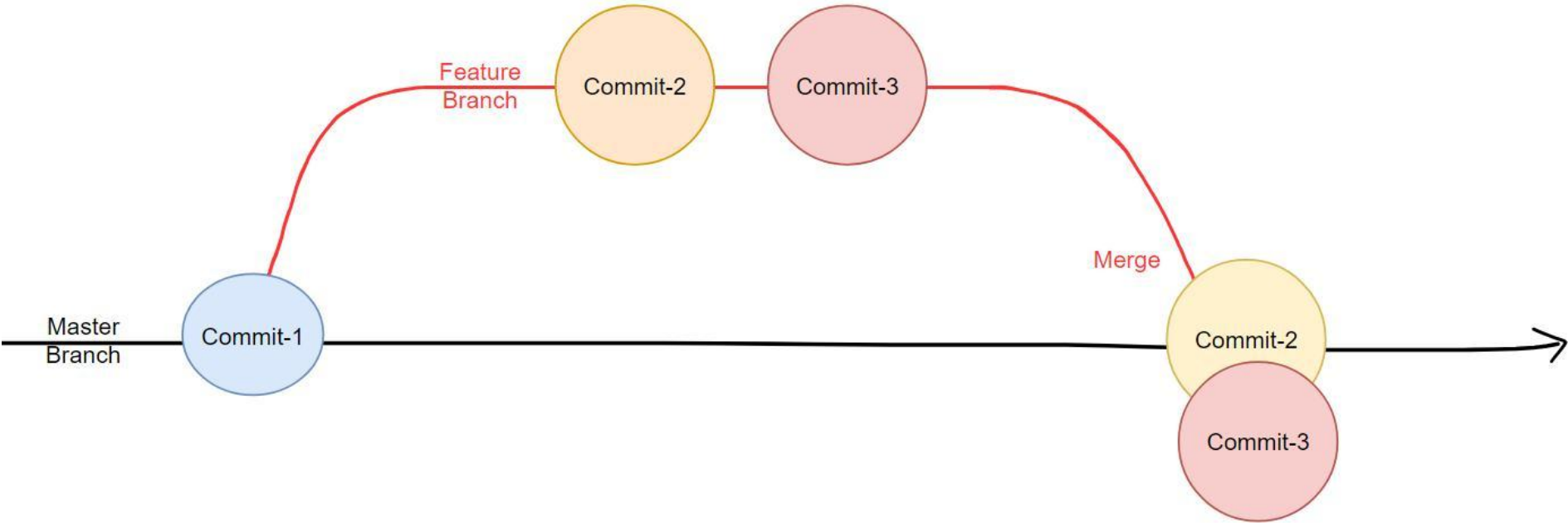


My Description: When doing the AZ-400 you might wonder why you need to re-create the Azure DevOps organisation for every lab. The reason is simply that Microsoft withdrew it's sponsorship of 'Azure Passes' and the companies that provide the labs need to work around that.



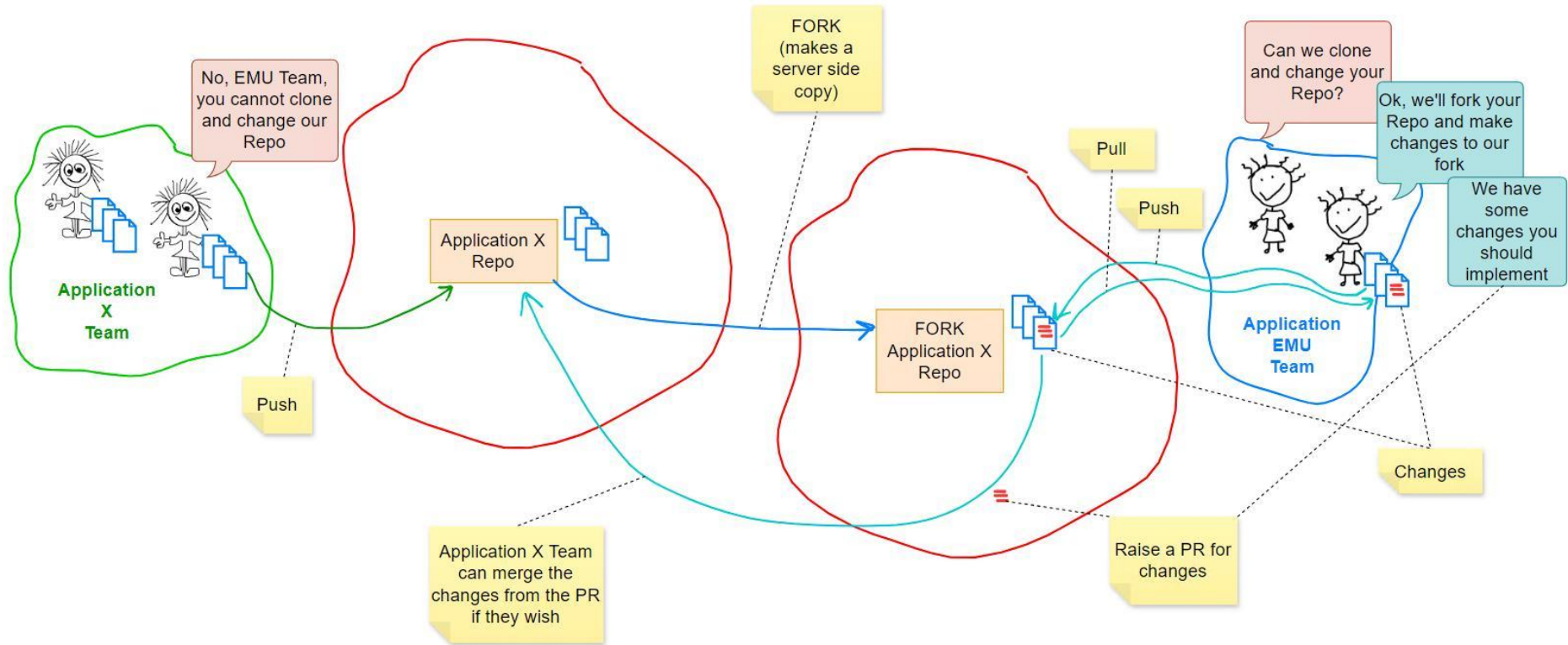
DiagramID: Skillable-Frustrating-Labs.

My Description: This is the essence of branching in repositories. A branch is a copy of the code at a point in time. Changes can be made to that branch without affecting other branches. When the changes are complete the branch can be merged back into another branch.



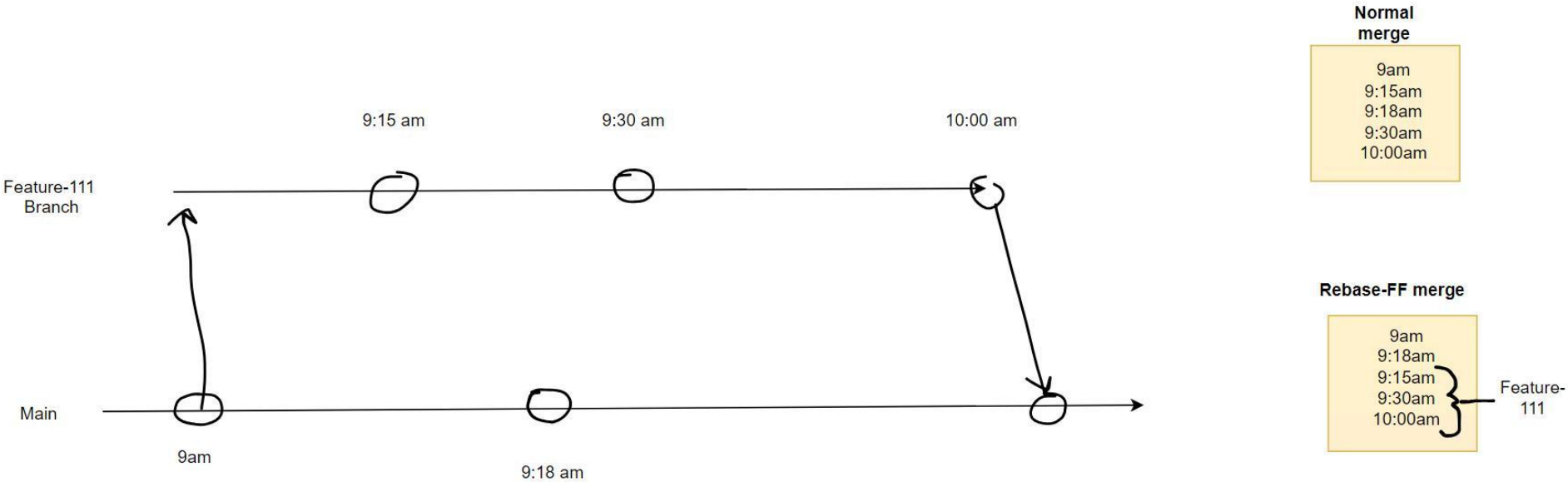
DiagramID: p-FeatureBranchBasic.

My Description: Forking in Git is typically used when members of an external team would like to copy a repo so that they can change or experiment with it. There is also a mechanism that external teams can use to have changes merged into the original repo via pull requests.



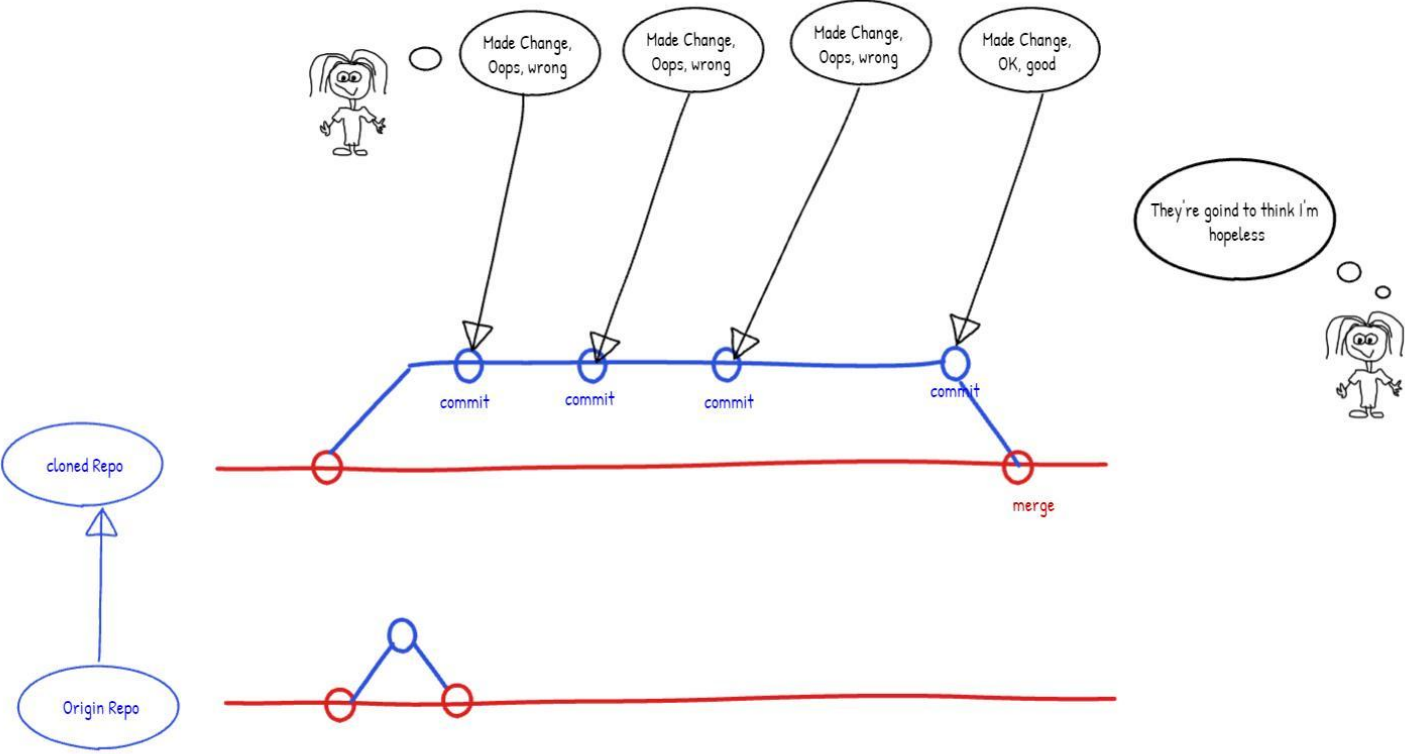
DiagramID: git-fork.

My Description: This shows 2 different merge strategies. Using a rebase and fast-forward approach keeps all the commits for a feature together



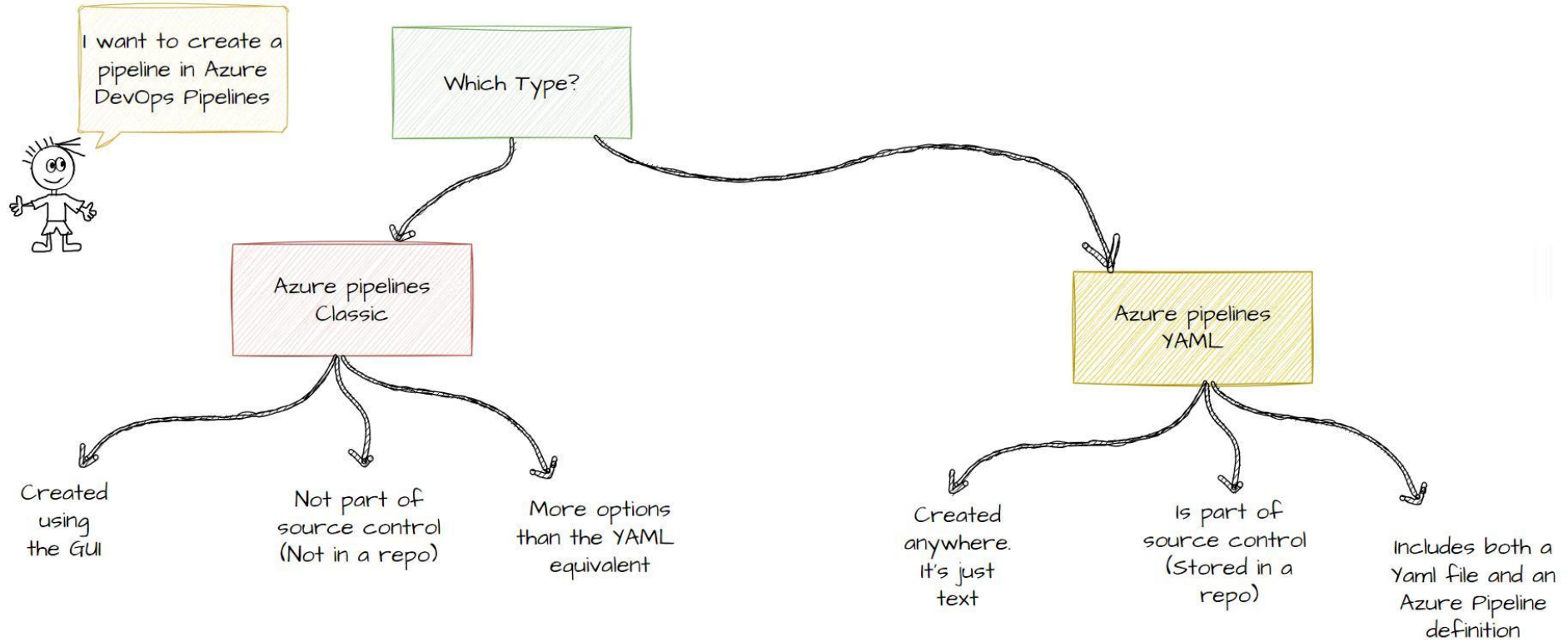
DiagramID: RebaseVsFFMerge.

My Description: Sometimes it makes sense to tidy up history



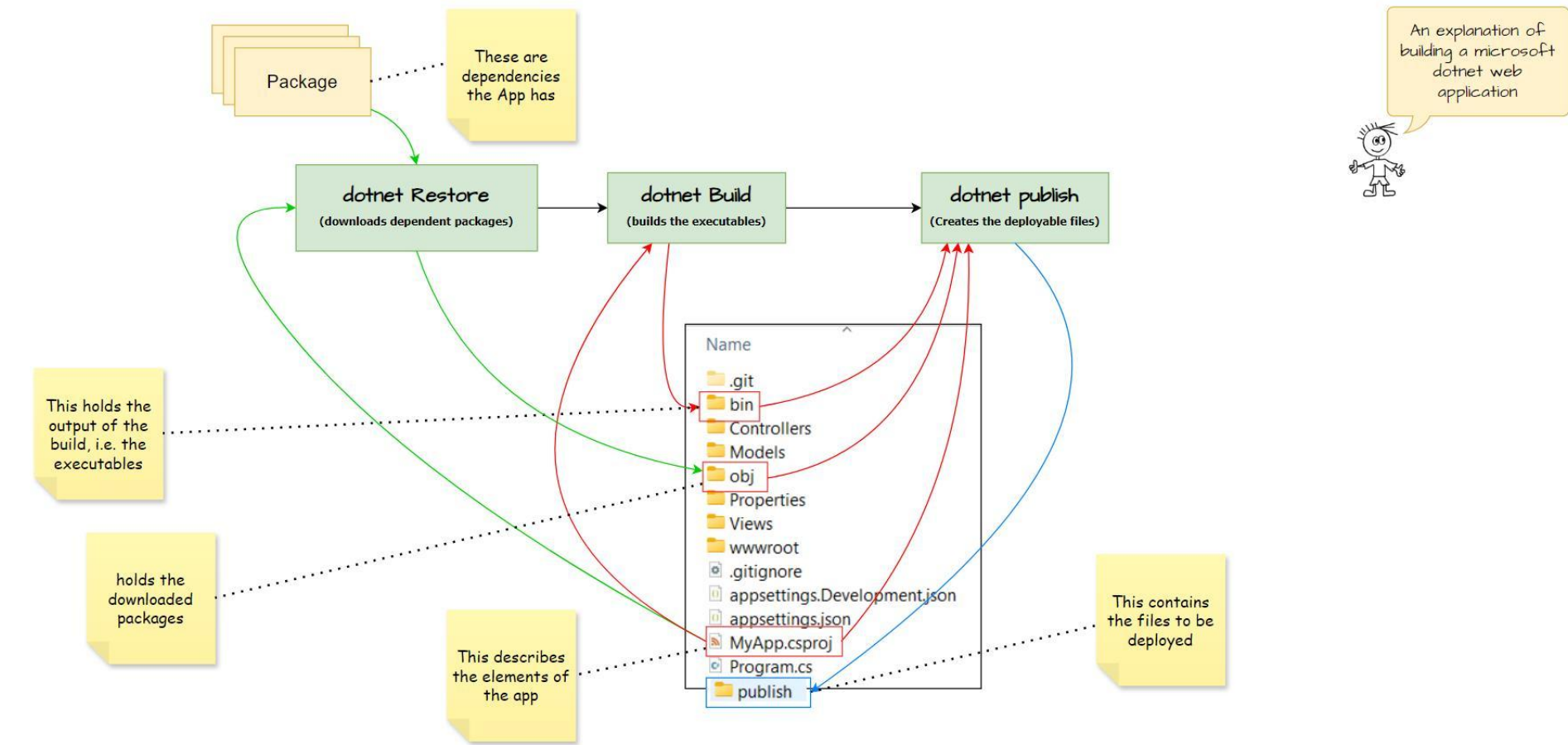
DiagramID: az-p-GitSquash.

My Description: When creating pipelines in Azure DevOps you have a choice of whether you design it using a Gui (Classic pipelines) or whether you design it with text (YAML pipelines). That's the case whether you're creating build pipelines or release pipelines. In this diagram I point out the main differences.



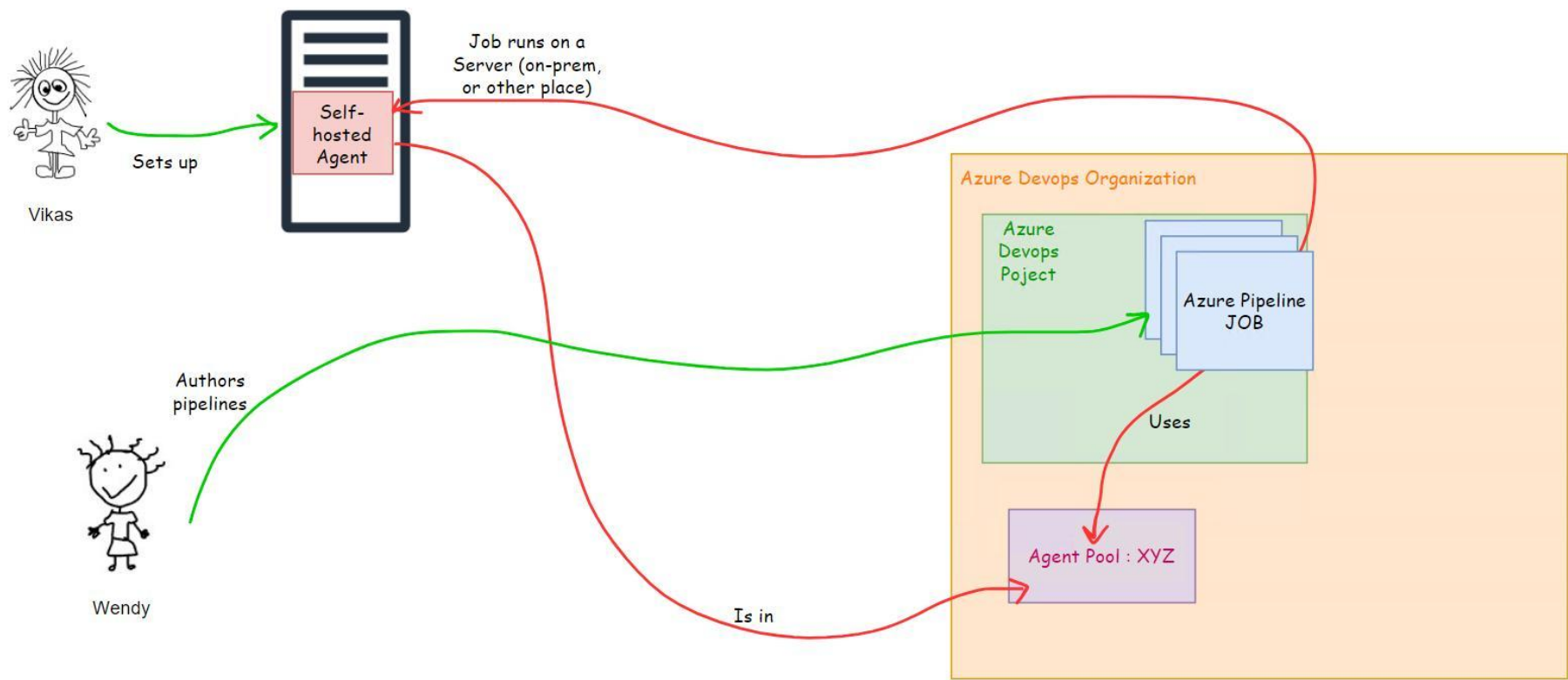
DiagramID: p-AzDevOpsPipelineTypes.

My Description: The purpose of this diagram is to describe a few key files that are in a .Net application and how they relate to each other. The diagram also shows the build and publish process that creates the files needed to run the application.



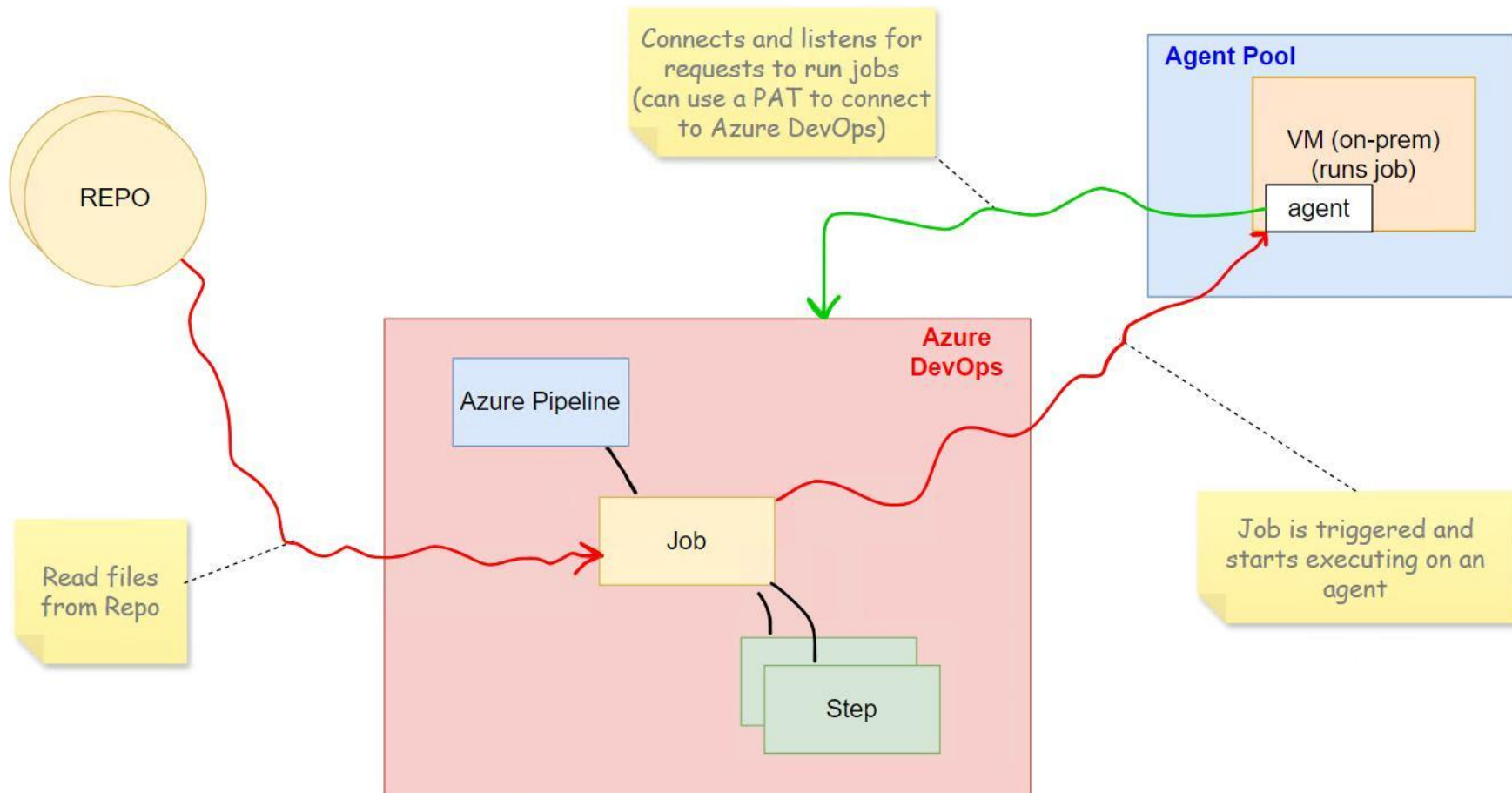
DiagramID: p-dotnet-build-publish.

My Description: The purpose of this diagram is to show the relationship between Azure Devops Jobs, Agent Pools, and Self-hosted agents.



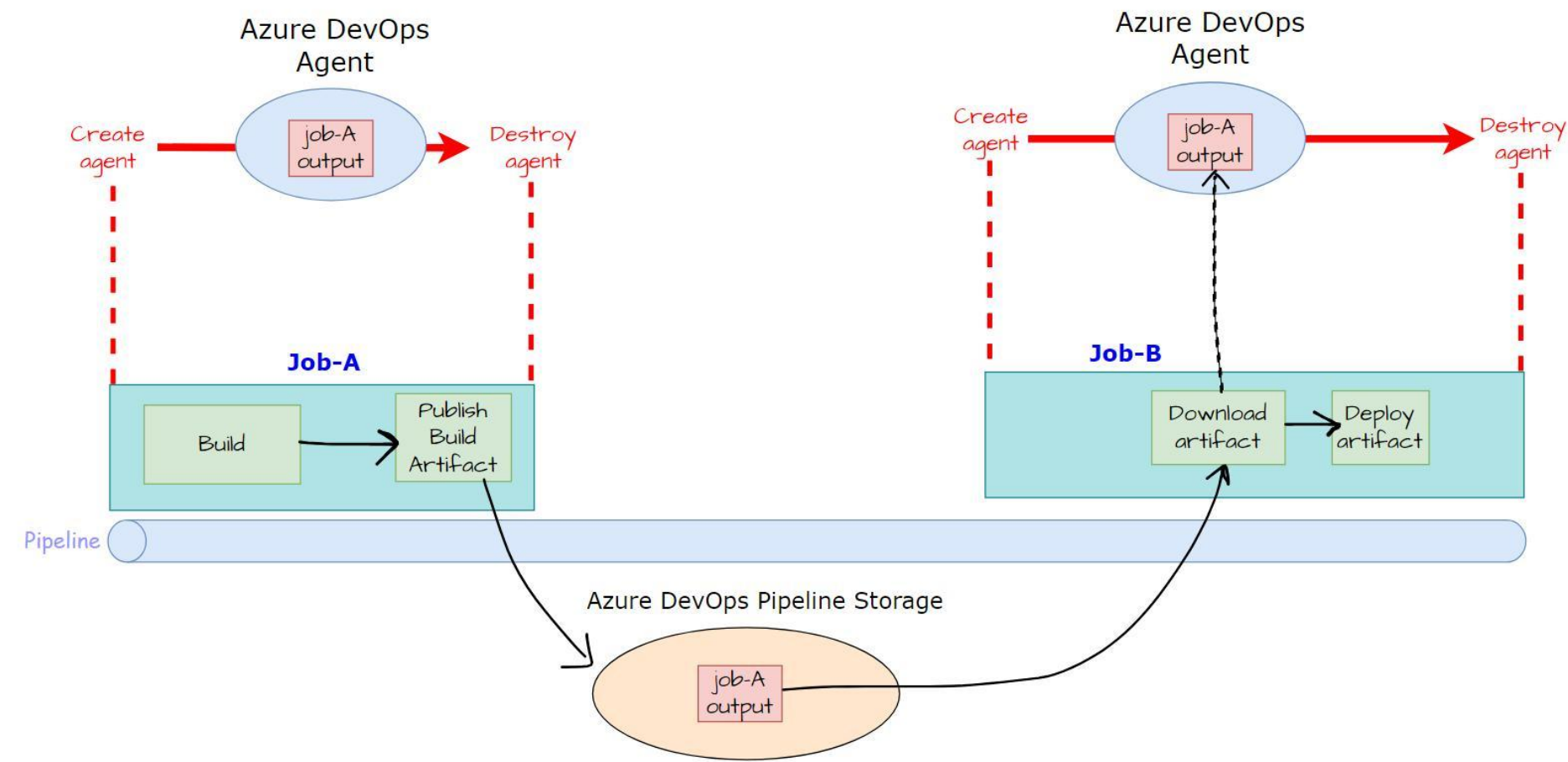
DiagramID: az-p-selfhosted-agents.

My Description: This diagram shows the architecture of Azure DevOps agent pools and agents. In this case the VM is one that we provision and manage. The agent is software downloaded from AzureDevOps and installed on the VM. The agent has to register itself with Azure Devops to be part of the Agent pool and needs to be authenticated to do so.



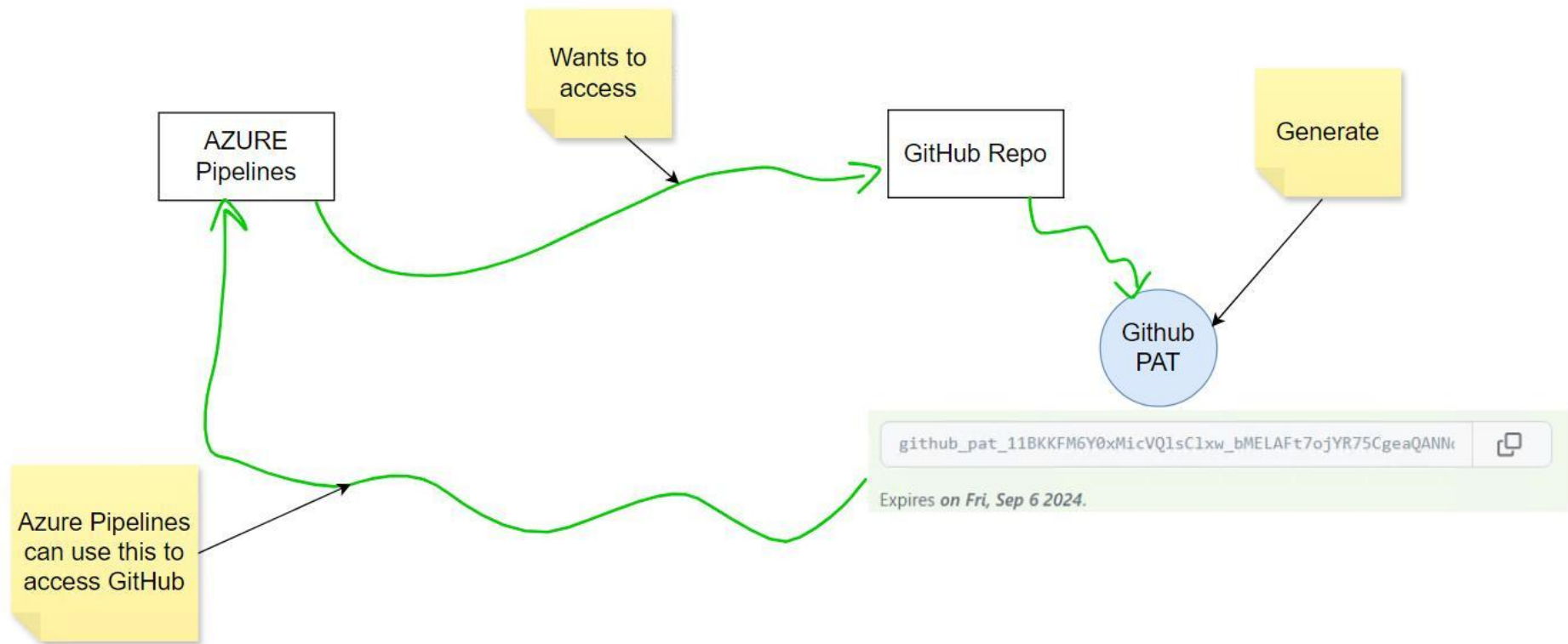
DiagramID: az-p-AgentPoolsAndAgents.

My Description: When a job runs in Azure DevOps it executes on an 'Agent'. That agent is a process running on a computer and is destroyed when the job completes; any files created as output would be destroyed would when the agent is destroyed. Consequently, a mechanism is needed to pass data between jobs and Azure pipeline storage can do that.



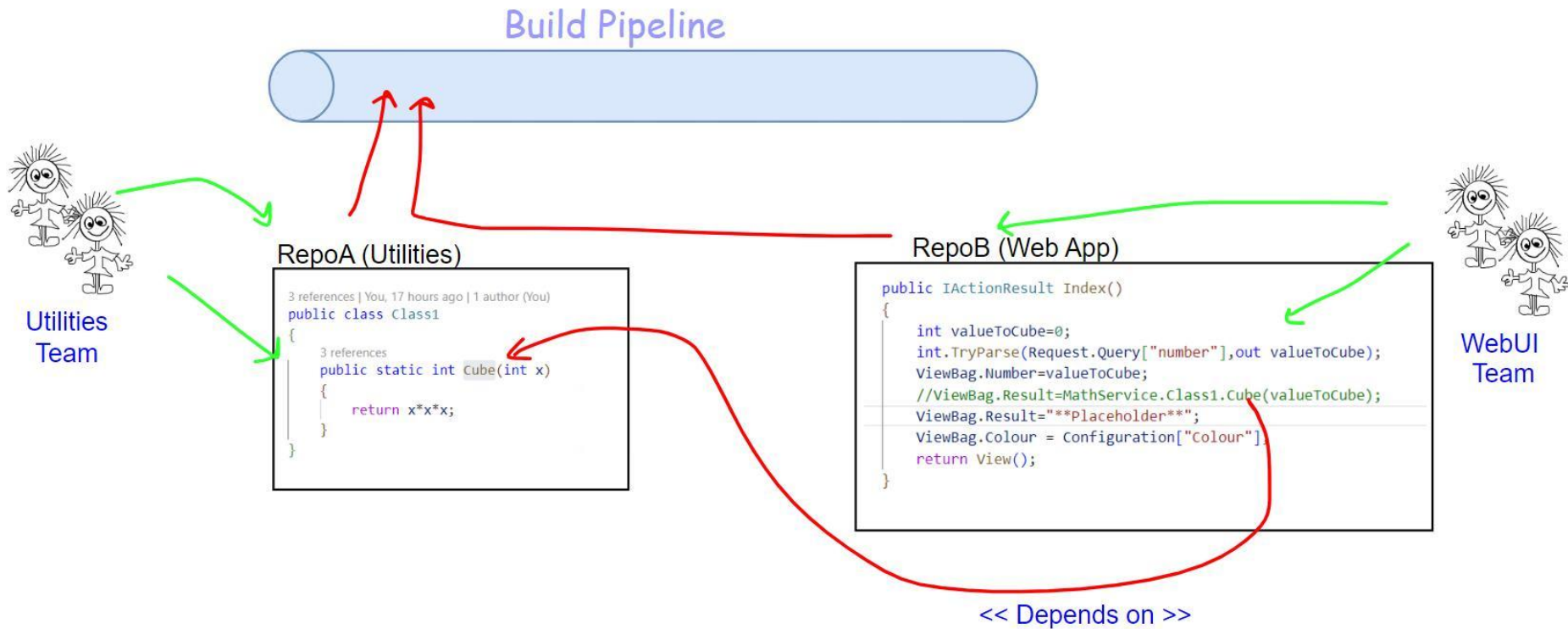
DiagramID: p-PassingArtifactsBetweenJobsAzDevOps.

My Description: If a pipeline running in Azure wants access to a private repository in Github then it needs some way to authenticate against Github. One of those ways could be a PAT (Personal Access Token) generated from github.



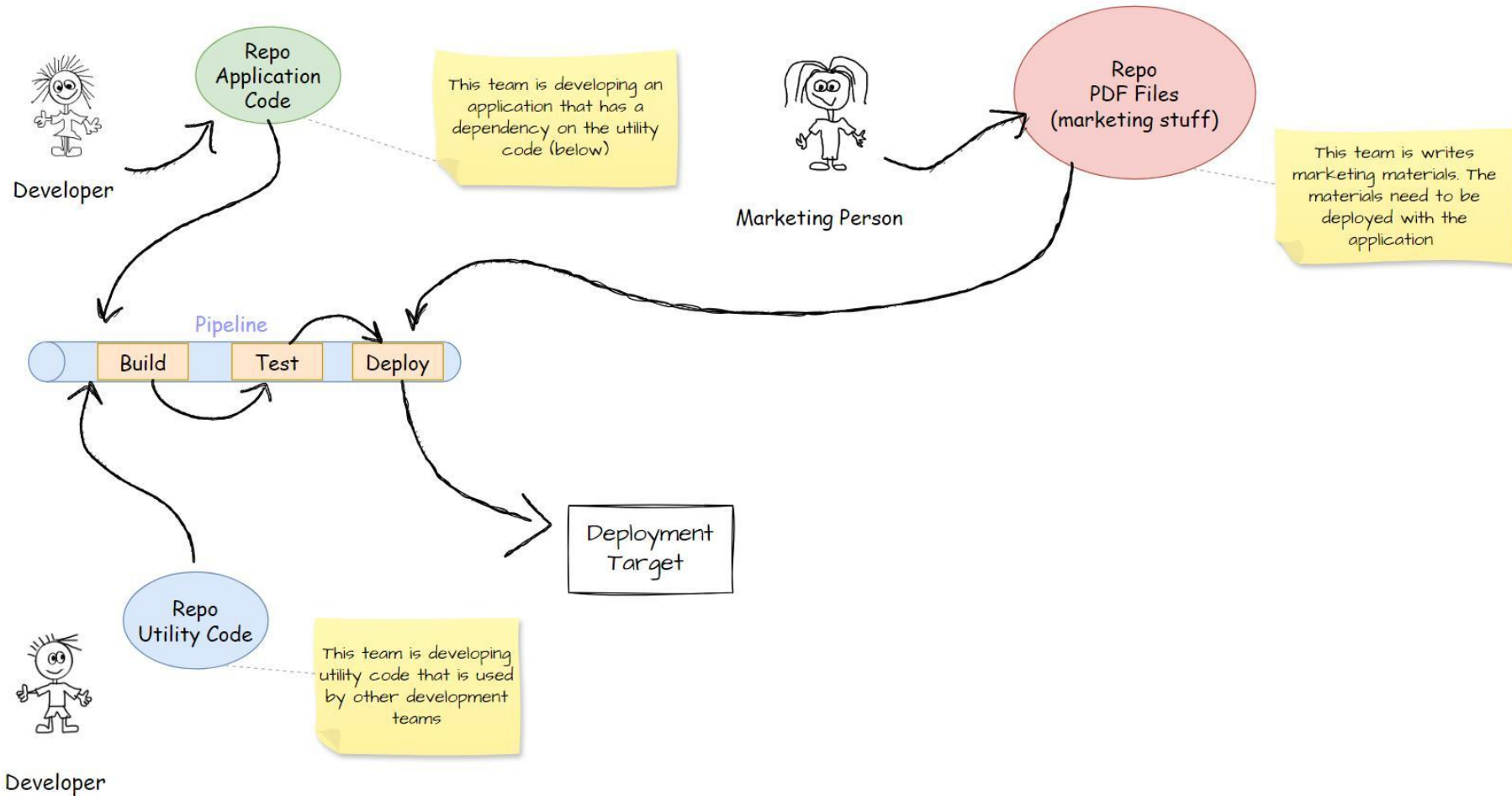
DiagramID: az-p-Github-PAT.

My Description: In this diagram there are 2 teams, they operate separately. Each looks after its own repo and writes its own code. The code in the Web Application depends on the code the utilities team writes. When the web application is to be deployed the pipeline has been designed to get the source code from both repo's.



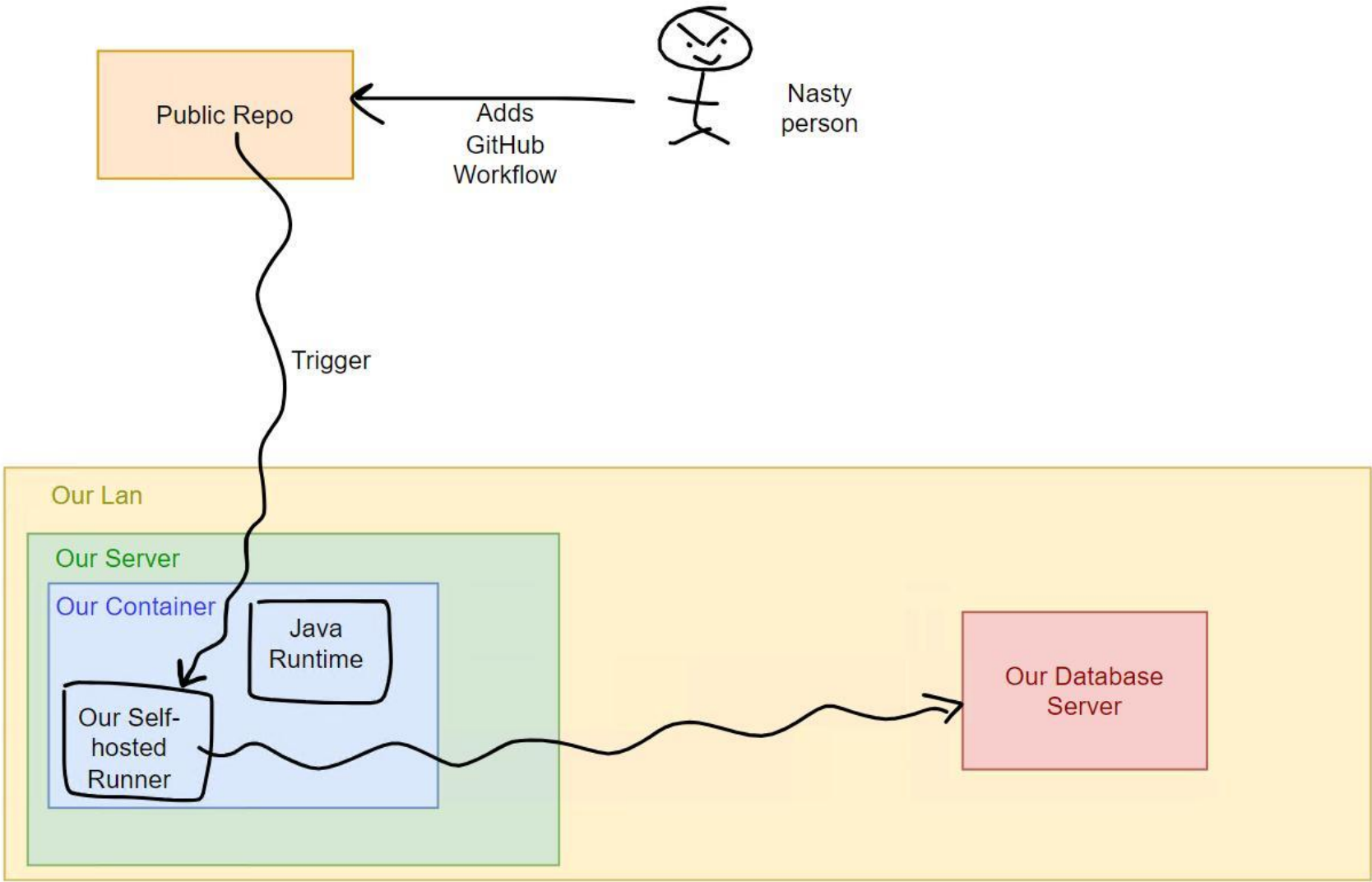
DiagramID: devops-p-multiplerepos.

My Description: In this diagram there are 3 teams, they operate separately. Each looks after its own repo and writes its own code (or content). The code in the Web Application depends on the code the utilities team writes and the web application needs the content from the marketing department. When the web application is to be deployed the pipeline has been designed to get the source code from all repo's.



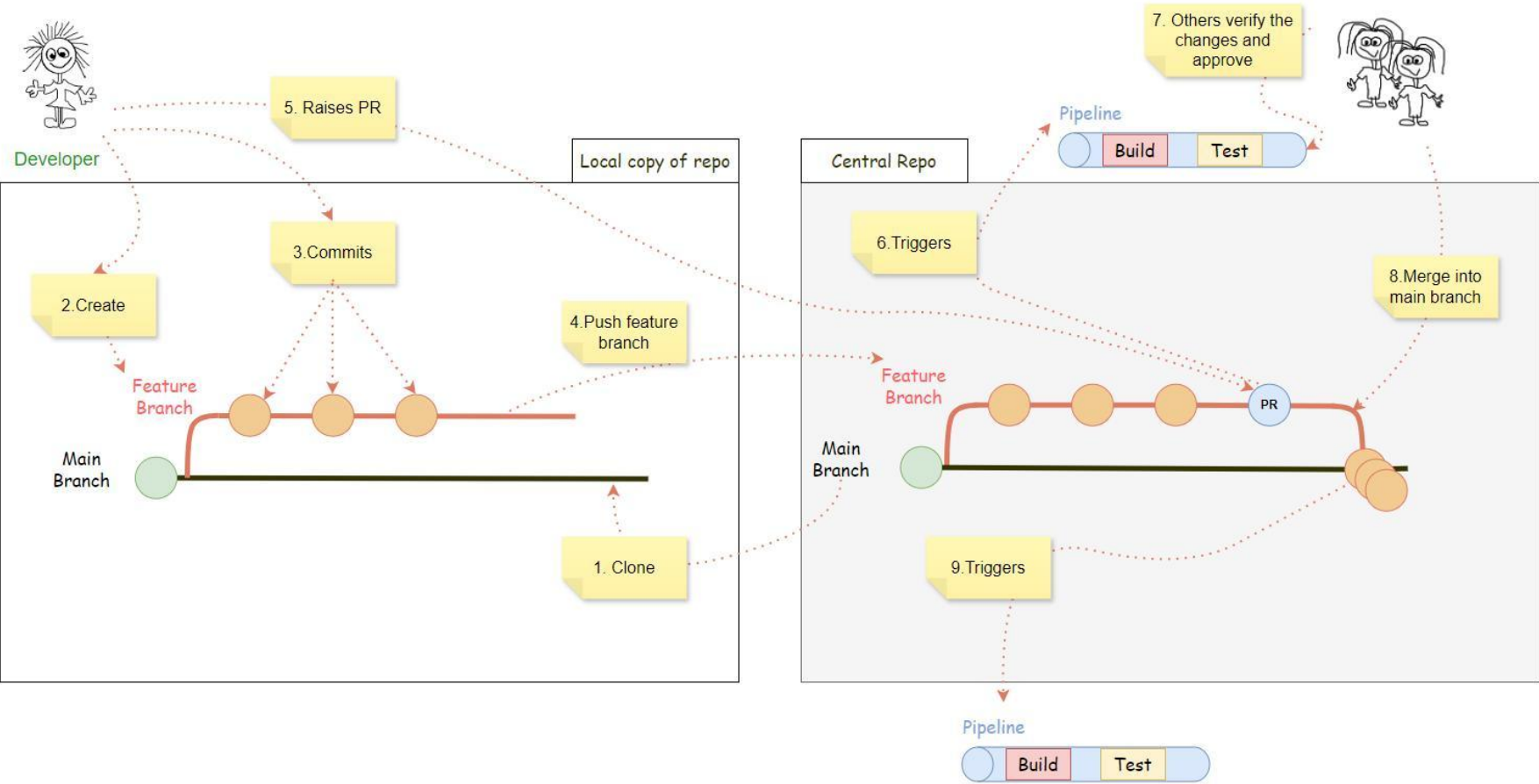
DiagramID: p-Multiple-Repos-In-Pipeline-2.

My Description: Having a self hosted runner triggered by a public repo can be risky. What if a nasty person works out a way to execute code on our on-premises server and that server has access to other on-premises resources with sensitive data?



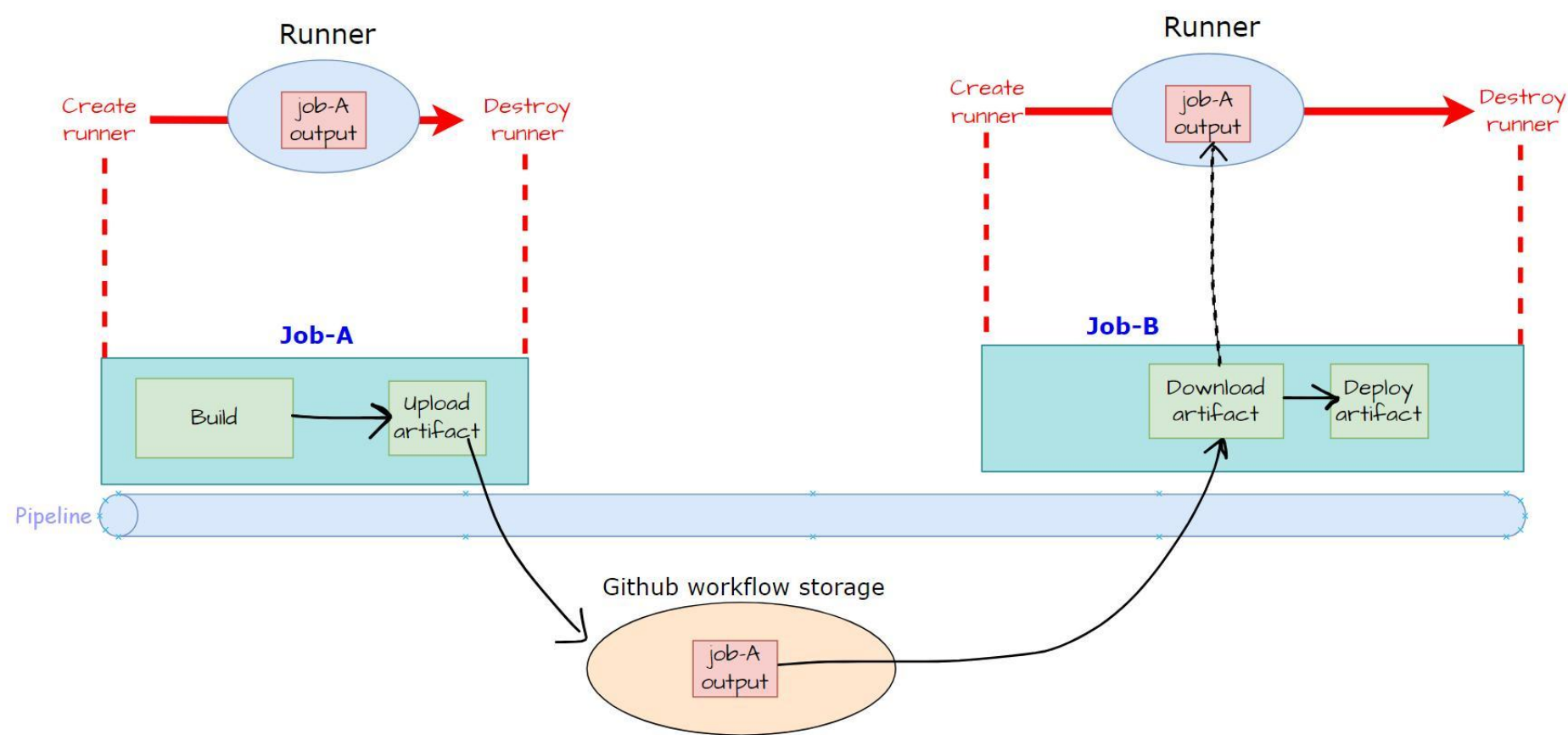
DiagramID: SelfHostedRunnerAndPublicRepo.

My Description: Continuous integration is a concept. Its implementation might be architected many different ways. Its goal is to get everybody's work into a central repository often, for that work to be high quality, and be in a state that is ready to deploy. In this diagram I'm showing what would be a common, simple implementation



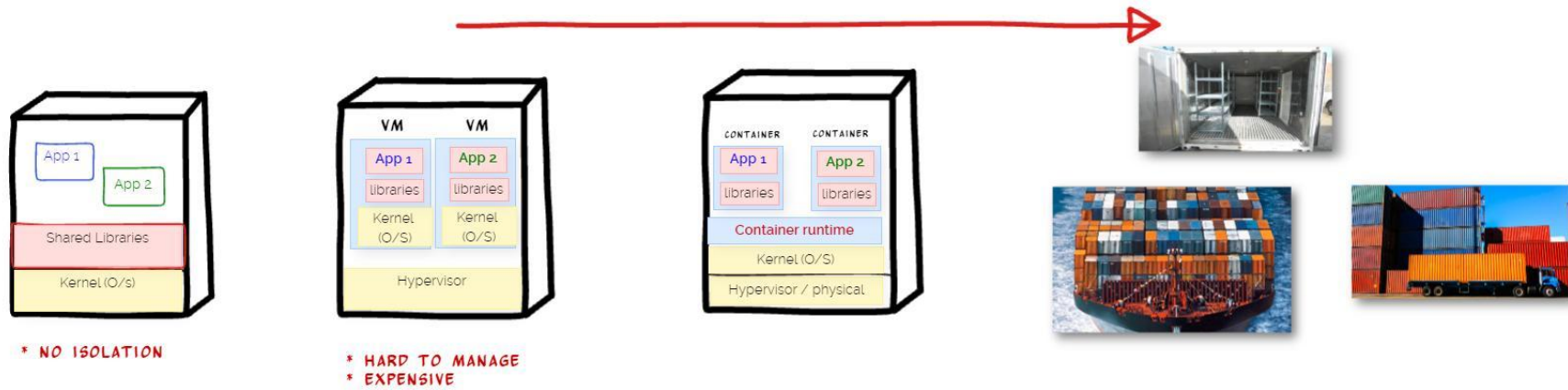
DiagramID: p-ContinuousIntegrationScenario.

My Description: When a job runs in GitHub it executes on an 'Runner'. That runner is a process running on a computer and is destroyed when the job completes; any files created as output would be destroyed when the runner is destroyed. Consequently, a mechanism is needed to pass data between jobs and GitHub pipeline storage can do that.



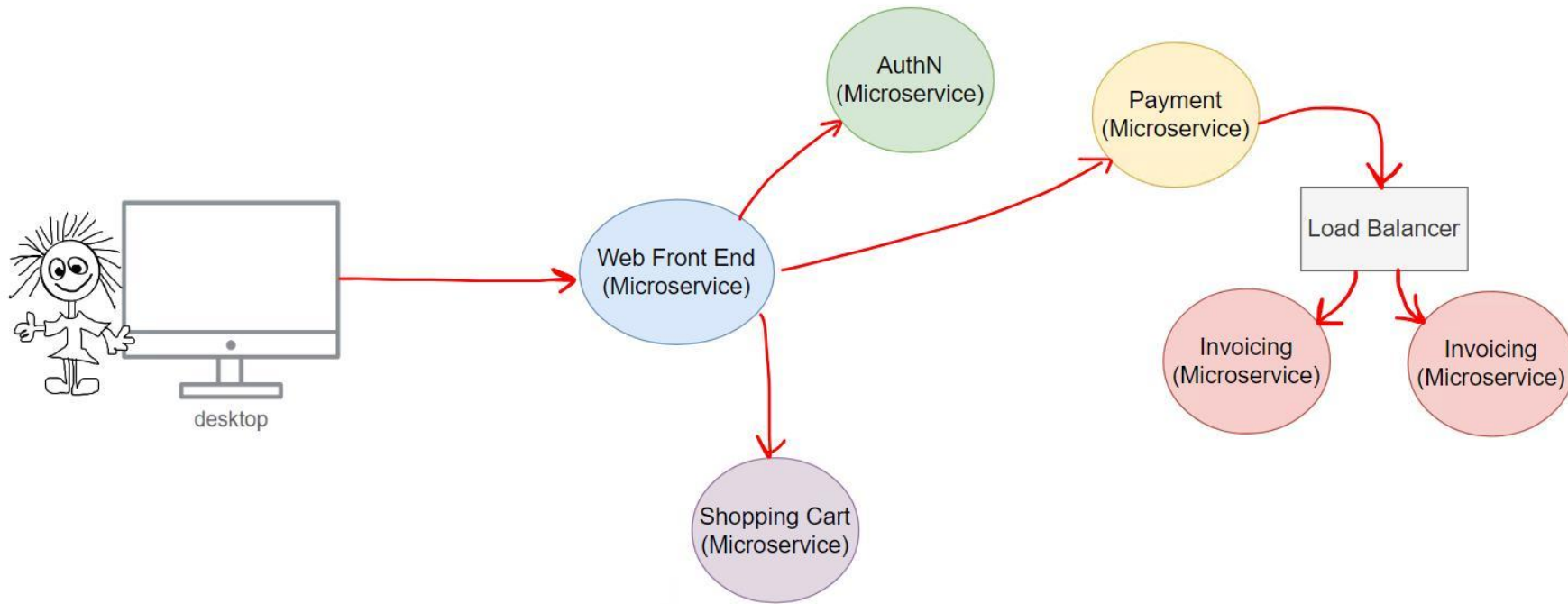
DiagramID: p-PassingArtifactsBetweenJobsGitHub.

My Description: Why Containers ? Before containers there was an issue where the deployment of software would cause another installed bit of software to fail, also software was often tightly coupled to a specific implementation of an operating System. Vm's can solve the problem but they are a poor choice in a Services type architecture where there could be 1000's of service running. Containers start and stop quickly and most importantly provide reliable deployment.



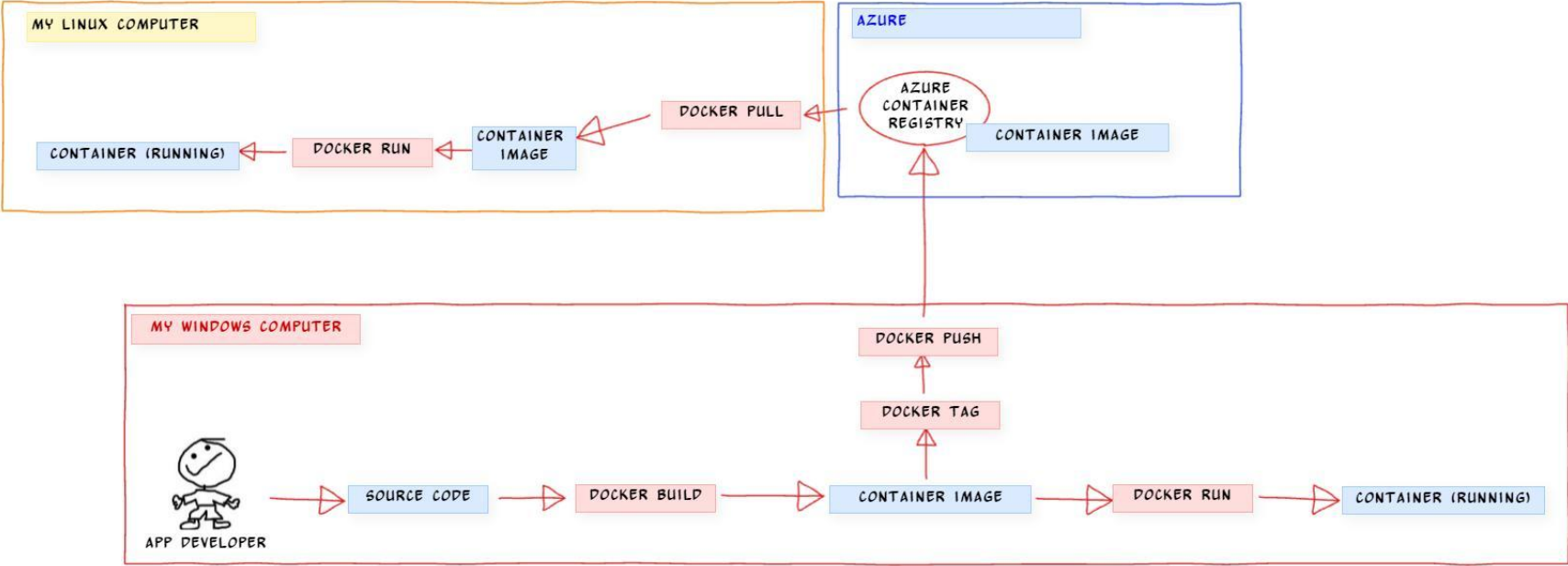
DiagramID: az-p-WhyContainers.

My Description: Here is a very basic diagram of an application architecture using Microservices. Each Microservice does one thing and does it well. Microservices call other microservices which might be on the same machine or other machines.



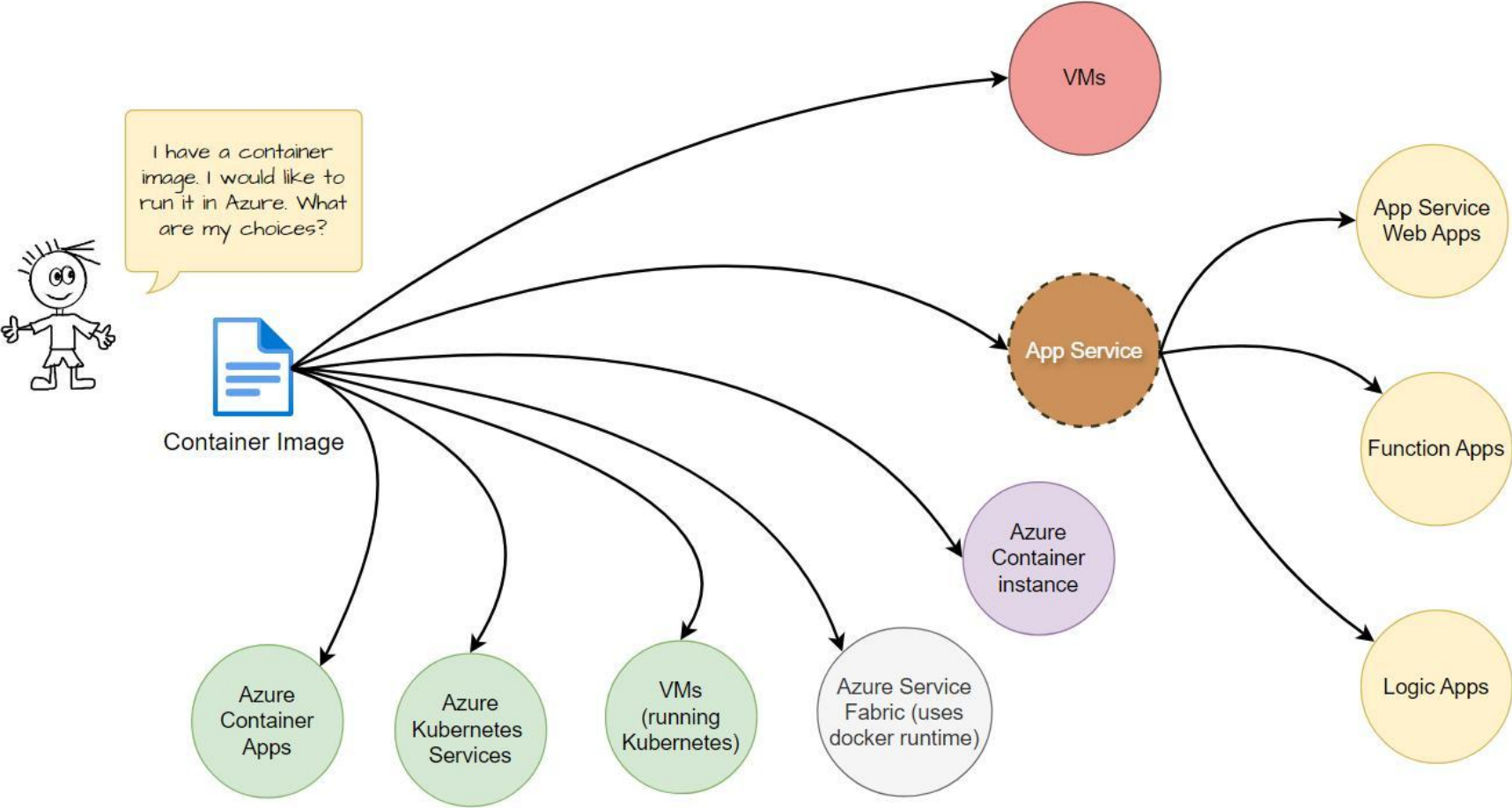
DiagramID: Microservices-Basic.

My Description: Lifecycle of a docker container



DiagramID: az-p-docker.

My Description: The choices we have for running containers in Azure



DiagramID: az-p-ContainerDeployChoices.